



MODUL 15 · DEPLOYMENT & PRODUCTION

Logging, Error Monitoring & Connection Pooling

Lihat apa yang terjadi di production. Tangkap error sebelum user lapor.



Vercel Logs & Structured Logging

Vercel punya built-in logs. Structured logging membuat logs mudah di-search dan di-filter.

```
// Structured logging – lebih baik dari console.log biasa
// Memudahkan search dan filter di production logs

type LogLevel = "info" | "warn" | "error" | "debug"

interface LogEntry {
  level:    LogLevel
  message:  string
  timestamp: string
  context?: Record<string, unknown>
}

function log(level: LogLevel, message: string, context?: Record<string, unknown>) {
  const entry: LogEntry = {
    level, message,
    timestamp: new Date().toISOString(),
    ...(context && { context }),
  }

  // JSON format: mudah di-parse oleh log aggregator
  console[level === "debug" ? "log" : level](JSON.stringify(entry))
}

export const logger = {
  info: (msg: string, ctx?: Record<string, unknown>) => log("info", msg, ctx),
  warn: (msg: string, ctx?: Record<string, unknown>) => log("warn", msg, ctx),
  error: (msg: string, ctx?: Record<string, unknown>) => log("error", msg, ctx),
  debug: (msg: string, ctx?: Record<string, unknown>) => {
    if (process.env.NODE_ENV !== "production") log("debug", msg, ctx)
  },
}

// Usage:
// logger.info("Post created", { postId: post.id, userId: session.user.id })
// logger.error("DB error", { error: e.message, query: "findUnique" })
```

Vercel Logs

Dashboard → Functions:

- Real-time logs
- Filter by level
- Search by text
- Retention: 1 hari (Hobby)
14 hari (Pro)

vercel logs --follow
(realtime CLI logs)

Log levels:

- info: normal operations
- warn: unexpected but ok
- error: needs attention

JANGAN log:

- Passwords
- API secrets
- Session tokens
- PII (nama, email, dll)

Log context yang berguna:
userId, postId, duration,
error code, route

Connection Pooling di Production

Sudah dibahas di Modul 10 Bab 8 — ringkasan checklist untuk production readiness.

Production database checklist

```
# — Database Production Checklist —————
# ✅ 1. Gunakan hosted database (bukan localhost)
DATABASE_URL="postgresql://...neon.tech/db?sslmode=require"

# ✅ 2. Dua URL: pooled (runtime) + direct (migrate)
DATABASE_URL="postgresql://...pooler.neon.tech/db?sslmode=require"
DIRECT_URL="postgresql://...ep-xxx.neon.tech/db?sslmode=require"

# ✅ 3. prisma.config.ts gunakan keduanya
import { defineConfig, env } from "prisma/config"
export default defineConfig({
  datasource: {
    url: env("DATABASE_URL"), // pooled untuk runtime
    directUrl: env("DIRECT_URL"), // direct untuk migrate
  }
})

# ✅ 4. lib/db.ts pakai adapter pg
# (sudah diajarkan di Modul 10 Bab 4)
import { PrismaPg } from "@prisma/adapter-pg"
const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL! })

# ✅ 5. Prisma client singleton dengan globalThis
# (sudah diajarkan di Modul 10 Bab 4)

# ✅ 6. SSL selalu enabled untuk hosted DB
# Sudah included di URL dengan ?sslmode=require
```

Monitoring DB

Neon Dashboard:

- Connection count
- Query duration
- Storage usage
- Branching untuk test

Prisma logging:

```
new PrismaClient({
  log: ["query", "error"]
})
```

→ log semua queries

Log slow queries:

Tambahkan timing ke

data fetching functions:

```
const start = Date.now()
const data = await db.post...
logger.info("query", {
  duration: Date.now()-start
})
```

Alert jika koneksi tinggi:

> 80% dari max connections

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Apa manfaat structured logging (JSON) dibanding console.log biasa?

- A) JSON lebih cepat di-eksekusi B) Mudah di-parse, di-search, dan di-filter oleh log aggregator. Consistent format. C) Hanya bisa pakai di production

Q2 — Kapan sebaiknya menggunakan `Sentry.captureException()` secara manual?

- A) Di setiap fungsi B) Di try/catch block untuk error yang butuh context tambahan (userId, action, dll) C) Hanya di page.tsx

Q3 — Mengapa `DATABASE_URL` dan `DIRECT_URL` harus berbeda di production?

- A) Hanya convention saja B) `DATABASE_URL`: pooled URL untuk runtime. `DIRECT_URL`: direct URL untuk prisma migrate (pooler tidak support DDL) C) Prisma require dua URL

Tugas Mandiri

Setup logging dan monitoring untuk production.

01 `lib/logger.ts`

Buat structured logger dengan JSON output. Replace semua `console.log` di Server Actions dengan `logger.info/error`. Tambahkan context yang berguna (`userId`, `postId`).

02 Sentry Setup

`bun add @sentry/nextjs`. Jalankan wizard. Set `SENTRY_DSN` di `.env`. Test: tambahkan `throw Error('test sentry')` di action → cek Sentry dashboard.

03 DB Monitoring

Buka Neon dashboard → Monitoring. Identifikasi peak connection time. Pastikan max connections tidak terlampaui saat load test.

04 Query Timing

Tambahkan timing logging ke 3 data fetching functions terpenting. Log: route, duration, row count. Identifikasi query yang > 100ms.



Sentry gratis: 5000 errors/bulan. Cukup untuk project awal. Upgrade jika butuh lebih.

Yang Sudah Kita Pelajari

- ✓ Structured logging (JSON): lebih mudah di-parse dan di-search dari plain console.log.
- ✓ Jangan log sensitive data: password, API secret, session token, PII.
- ✓ Sentry: error monitoring otomatis + stack trace + notifications. Gratis 5000 errors/bulan.
- ✓ DATABASE_URL (pooled) vs DIRECT_URL (direct): keduanya wajib di production dengan pooler.
- ✓ Monitor koneksi DB: alert jika mendekati max connections limit.

→ Selanjutnya: **Bab 5 — Security Checklist Production**

